

Universidade Federal de Lavras
PPGCC
PCC508 – Sistemas Operacionais
Lista Avaliativa – 5

Exercícios sobre gerência de memória:

1) Poderíamos criar uma ferramenta para analisar cada um dos programas para prever quando cada página seria utilizada, de modo a implementarmos o algoritmo de substituição de páginas ótimo? Explique.

2) Explique o algoritmo de substituição de páginas Not Recently Used (não usada recentemente), apresentando um exemplo.

3) Considere o algoritmo de substituição de página “Least Recently Used” (LRU) implementado com um hardware que mantém uma matriz de $n \times n$ bits. No exemplo, n será de 4 bits. Agora imagine a seguinte ordem de referência: 3 3 2 1 2 3 0. Mostre a sequência de matrizes de acesso do LRU para a ordem dada. Qual página que seria escolhida para substituição, se fosse necessário retirar uma?

4) O algoritmo PFF (Page Fault Frequency – Frequência de Falta de Página) é utilizado para controlar o tamanho do conjunto de páginas alocadas na memória RAM de um determinado processo, quando um algoritmo de alocação global é utilizado. Explique, com exemplos, como funciona este algoritmo.

5) Explique como funciona a tradução de um endereço MULTICS para o endereço físico da máquina. O MULTICS trabalha com segmentação com paginação. Explique como funciona o descritor de segmento juntamente com as tabelas de páginas.

Exercício de implementação sobre chamadas de sistema:

6) Nos dias atuais, sistemas com múltiplos cores dentro do processador estão em toda a parte. No Linux, podemos determinar a afinidade de uma determinada thread a um dado core com a função: *sched_setaffinity*. Faça um programa que gere 4 vetores de tamanho n com números aleatórios. Cada vetor deve ser ordenado por uma thread diferente. As threads devem ser divididas entre os cores existentes e cada thread deve sempre ser executada no mesmo core. Você deve verificar se a afinidade da thread foi escolhida corretamente com a função *sched_getaffinity*. Além disso, o comando *top* deve ser utilizado para a verificação do core que executa cada thread.